



Informe Técnico

Maquina: OWASPBWA



Este documento es confidencial y contiene información sensible.
No debería ser impreso o compartido con terceras entidades

11 de Febrero de 2025



Índice

1. Antecedentes	2
2. Objetivo	2
2.1. Alcance	3
2.2. Impedimentos y limitaciones	3
2.3. Resumen general	3
3. Reconocimiento	4
3.1. Enumeración de servicios expuestos	4
3.2. Enumeración de servidores web	5
4. Identificación y explotación de vulnerabilidades	7
4.1. Explotación de vulnerabilidad XSS	7
4.1.1. Explotación gráfica usando alert()	7
4.1.2. Explotación gráfica obteniendo la cookie	8
5. Contramedidas y buenas practicas	9
5.1. Vulnerabilidad XSS	9
6. Conclusión	9



1. Antecedentes

El presente documento recoge los resultados obtenidos durante la fase de auditoría realizada a la maquina **OWASPBWA**, enumerando todos los vectores de ataques encontrados, así como la explotación realizada para cada uno de estos dentro del apartado **A3. Cross-Site Scripting (XSS) / Cross-Site Scripting - Reflected (GET)**.

Esta maquina ha sido descargada desde la plataforma **SourceForge**, una plataforma para el desarrollo y distribución de software de código abierto..

A continuación, se proporciona el enlace directo a la máquina:

Dirección URL

<https://sourceforge.net/projects/owaspbwa/>

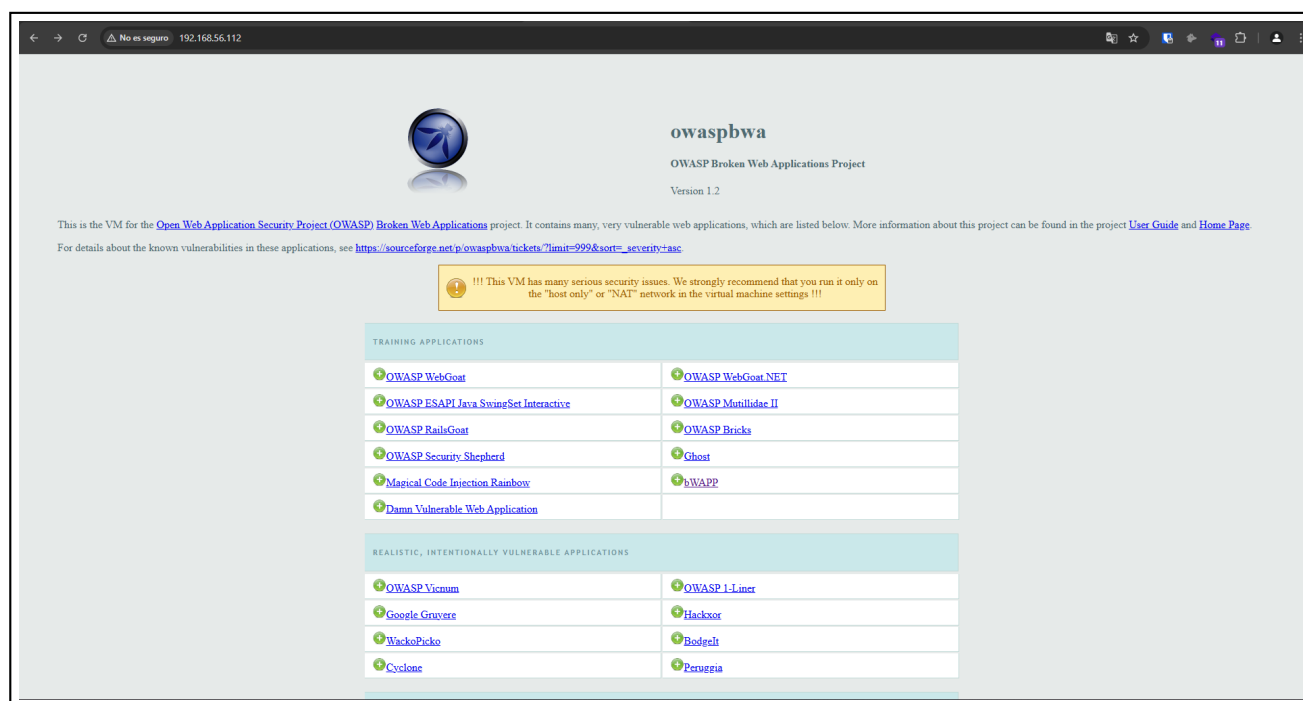


Imagen 1: Pagina principal del servicio web que proporciona la maquina

2. Objetivo

Los objetivos de la presente auditoria se enfocan en la identificación de posibles vulnerabilidades y debilidades en la maquina **OWASPBWA**, en el apartado **A3. Cross-Site Scripting (XSS) / Cross-Site Scripting - Reflected (GET)**, con el propósito de garantizar la integridad y confidencialidad de la información almacenada en ella.

Con este fin se ha llevado acabo un análisis exhaustivo de todos los servicios detectados que se encontraban en dicho servidor, recopilando información detallada de aquellos que representan un riesgo potencial desde el punto de vista de la seguridad informática.





3. Reconocimiento

3.1. Enumeración de servicios expuestos

A continuación, se adjunta una evidencia del contenido que muestra el servicio web que vamos a auditar:

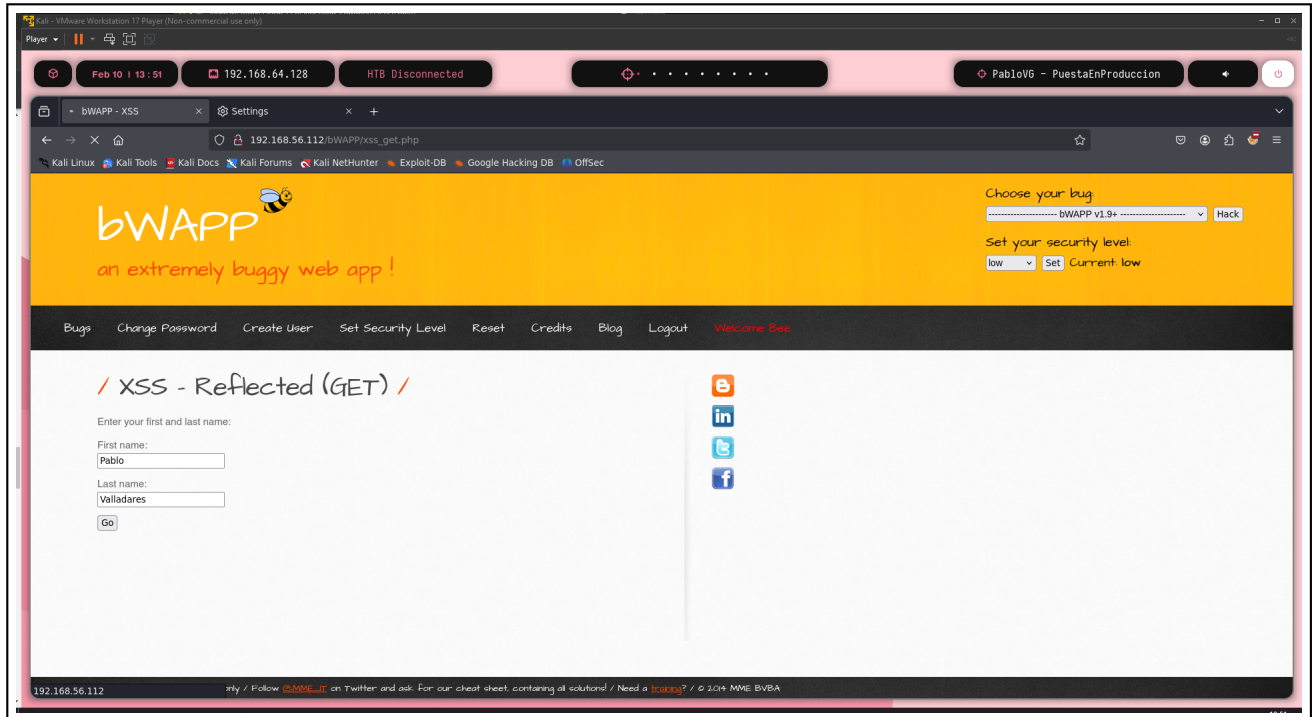
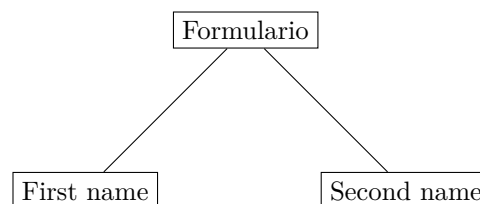


Imagen 3: Muestra gráfica del contenido de la pagina

En este caso, se identificaron 2 campos en los cuales se puede introducir contenido en dicho formulario:



Asimismo, se pudo apreciar que el servicio web dice tener un formulario de tipo **GET**, por lo que también aparecen en la URL. En caso de que esto sea cierto, este es otro método de introducir datos validos al formulario una vez conozcamos la estructura de la petición.



3.2. Enumeración de servidores web

A continuación, se representan los resultados obtenidos con la herramienta **burpsuite**, una herramienta que permite interceptar las solicitudes y respuestas entre el navegador y el servidor, revelando así información valiosa sobre la infraestructura del sitio, tras aplicar un reconocimiento sobre el formulario en el servicio HTTP corriendo en el puerto 80:

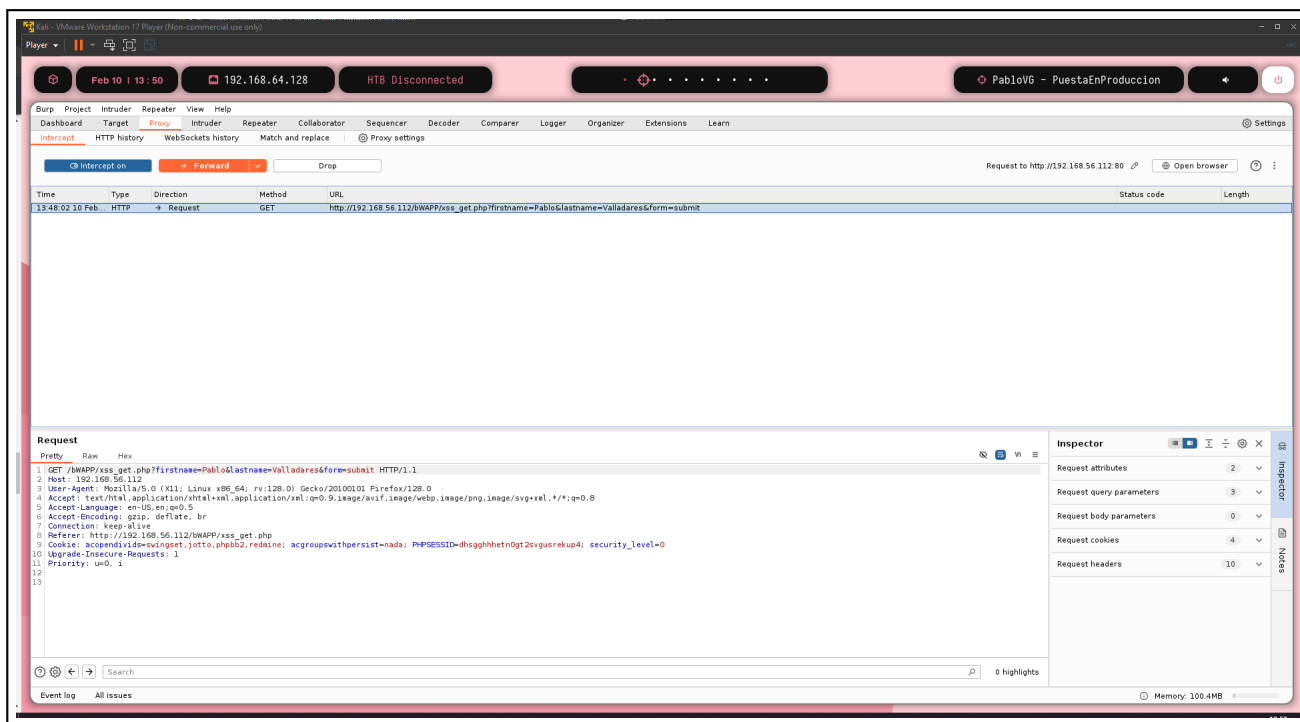


Imagen 4: Representación de la petición GET interceptada

En los resultados obtenidos, es posible identificar que la petición es ciertamente de tipo **GET**, además de que su estructura es:

Petición GET enviada

```
GET /bWAPP/xss_get.php?firstName=Pablo&lastName=Valladares&form=submit HTTP/1.1
```

Por lo que se puede apreciar tanto en la respuesta del servidor como en la petición enviada, la petición no ha sido sanitizada y tampoco vemos una clara evidencia de que haya sido debido a que la petición no tenga ningún elemento peligroso o inadecuado. Como consecuencia nos disponemos a introducir una petición con contenido **HTML** siendo esta:

Campo	contenido introducido
First name	PRUEBA
Last name	prueba

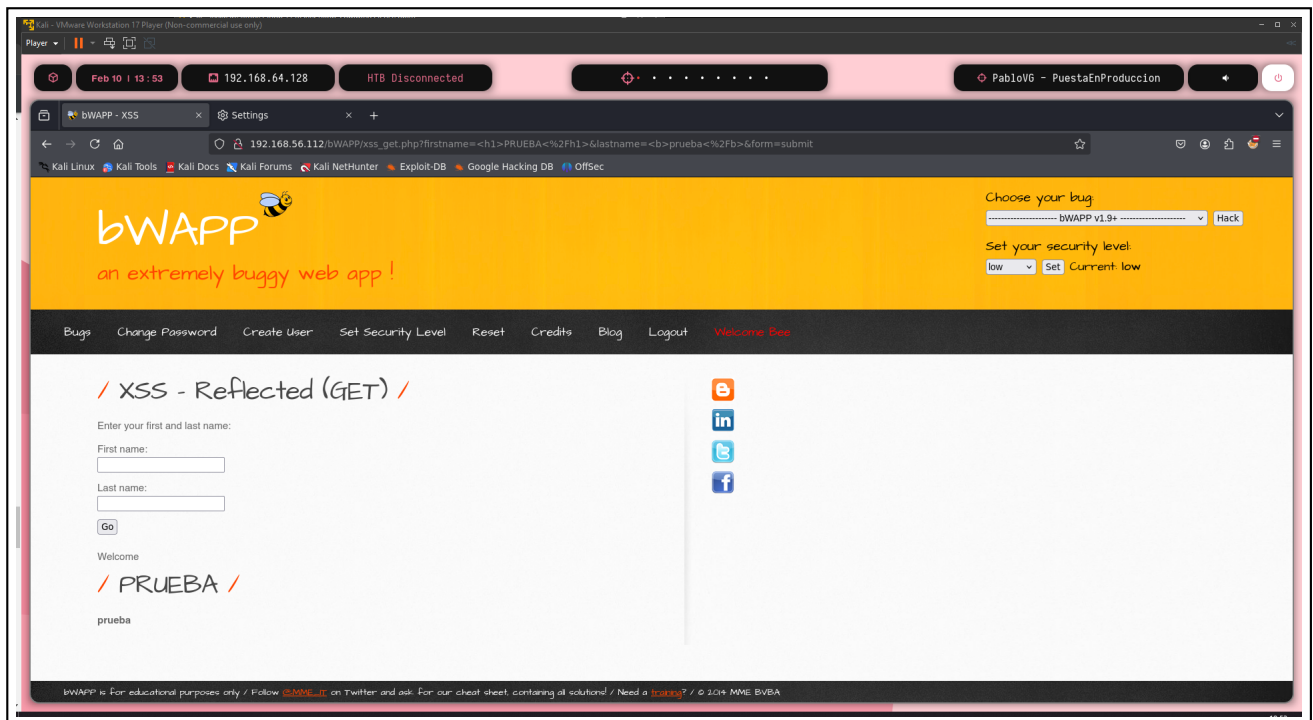


Imagen 5: Representación de la petición GET interceptada

Tal y como se aprecia en la imagen, la pagina es susceptible a inyecciones de tipo **HTML** ya que ha ejecutado el input como codigo en vez de como texto plano, indicando esto que podemos introducir codigo HTML para ejecutar **JavaScript** y realizar un ataque de tipo **XSS**.



4. Identificación y explotación de vulnerabilidades

4.1. Explotación de vulnerabilidad XSS

Gracias a la labor de reconocimiento previamente realizada, sabemos que el formulario en nuestro servicio objetivo es susceptible a **inyecciones XSS**, por lo que con ayuda de la herramienta **burpsuite**, previamente explicada en este mismo documento. Se realizaron múltiples pruebas de XSS, siendo algunas de las más destacadas las siguientes:

4.1.1. Explotación gráfica usando alert()

A diferencia del caso utilizado como prueba para la **inyección HTML**, en estos casos solamente vamos a utilizar el campo de **First name**, ya que al tener el mismo nivel de sanitización en ambos campos, lo que funcione en uno funcionara en el otro y viceversa. Para realizar un ataque de **XSS** se pueden utilizar diferentes métodos, ya sea la etiqueta script u otras formas, en nuestro caso se va a precisar de la etiqueta script como primera toma de contacto, en caso de que no funcionase utilizaríamos otros métodos.

Campo	contenido introducido
First name	<script>alert("dentro")</script>
Last name	..

Dando como resultado la activación de dicha alerta en **JavaScript**, asegurando y confirmando la posibilidad de realizar un ataque de tipo **XSS**

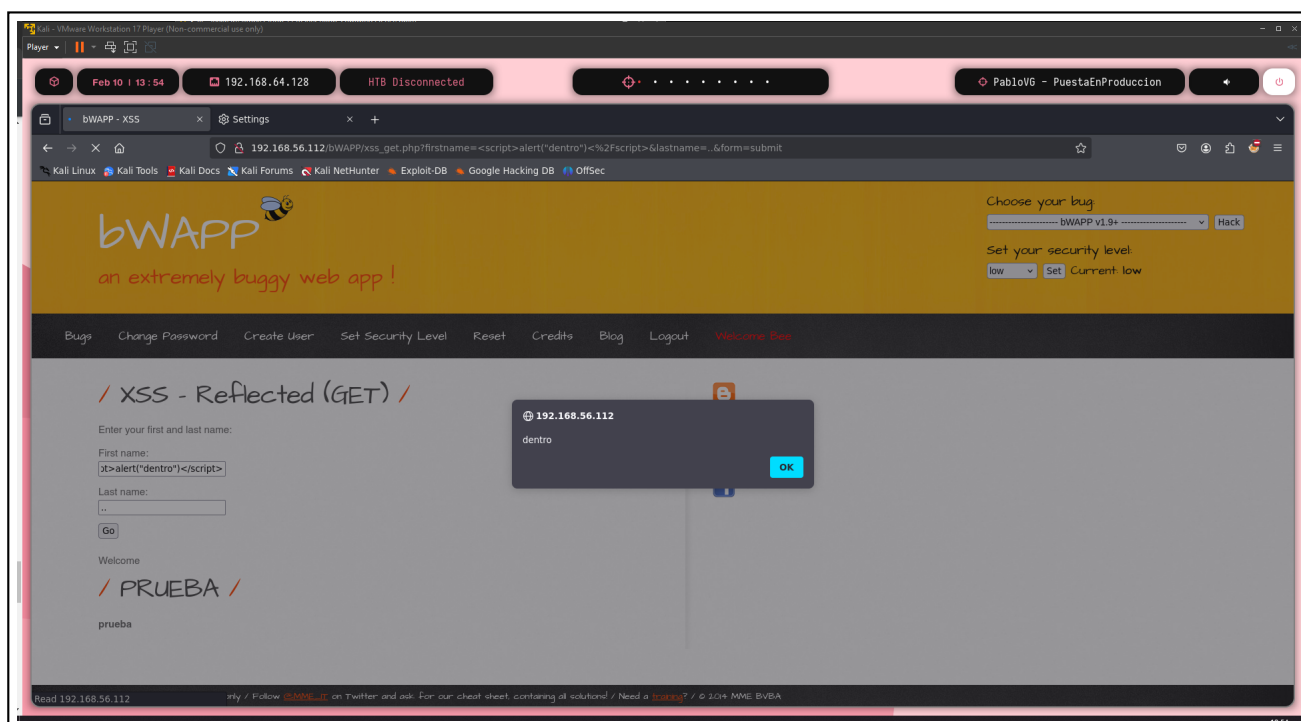


Imagen 6: Resultado de la explotación XSS con alert()



4.1.2. Explotación gráfica obteniendo la cookie

Con un método ya establecido de como generar un XSS en dicho servicio web, más concretamente en el formulario de la pagina en el puerto 80, podemos diseñar una petición la cual nos devuelva/muestre la cookie de sesión de dicho usuario, para eso tendremos que utilizar los siguientes inputs

Campo	contenido introducido
First name	<code><script>alert(document.cookie)</script></code>
Last name	..

Dando como resultado la activación de dicha alerta en **JavaScript** mostrándonos la cookie, asegurando y confirmando la posibilidad de robar la cookie de sesión a un usuario mediante la divulgación de la URL. Aunque la URL actual no hace más que mostrarnos a nosotros mismo la cookie, si añadiéramos scripts más maliciosos como podrían ser el envío de dicha cookie a un servidor de tercero para que este pueda utilizarla, seria bastante perjudicial

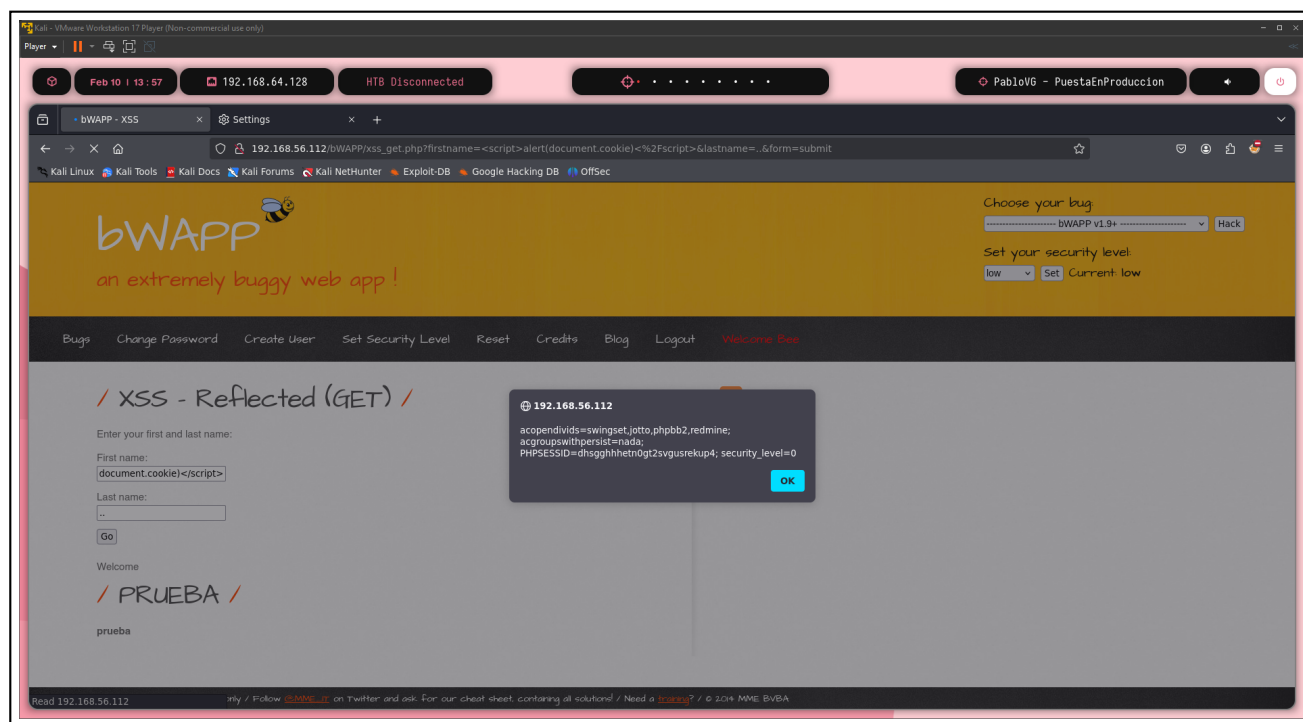


Imagen 7: Resultado de la explotación XSS con alert() mostrando la cookie de sesión



5. Contramedidas y buenas practicas

Con el objetivo de evitar posibles explotaciones indeseadas en el servidor expuesto, se enumeran a continuación las buenas prácticas a llevar a cabo para las diferentes vulnerabilidades descubiertas.

5.1. Vulnerabilidad XSS

Siendo esta la principal vulnerabilidad expuesta en el servidor web en el puerto 80 y siendo además la única encontrada en dicho apartado objetivo de la auditoria.

Como contramedida a dicha vulnerabilidad se recomienda una sanitización de la entrada de datos, obligatoriamente en el servidor, aunque se puede añadir también en el cliente pero esta simplemente sería inútil si no hay ninguna validación extra en el servidor, ya que las validaciones en el cliente pueden llegar a ser modificadas.

Como medida de sanitización se recomiendan algunos posibles ejemplos:

- Evitar el uso de peticiones GET, utilizando peticiones POST u otro tipo.
- Reemplazar todo tipo de etiqueta HTML por vacío o en su incapacidad mostrarlas como texto plano.
- Evitar el uso de formularios o cualquier tipo de input del usuario a no ser que sea estrictamente necesario.

Como información extra a las recomendaciones se explican de una forma más detallada:

A diferencia de **GET** (Los parámetros de la solicitud se envían en la URL, lo que significa que son visibles y pueden ser almacenados en el historial del navegador.), las peticiones **POST** envían datos en el cuerpo de la solicitud, lo que las hace más adecuadas para enviar información sensible o grande. Esto reduce el riesgo de exposición de datos en la **URL** y permite enviar datos más complejos, como archivos o formularios.

En caso de reemplazar el texto introducido por el usuario se podría hacer utilizando la propiedad **textContent** en JavaScript, que automáticamente escapa cualquier **HTML**.

Limitar el uso de **formularios** e inputs solo a aquellos casos donde sea absolutamente necesario ayuda a reducir la superficie de ataque. Si no se necesita recopilar información del usuario, es mejor no incluir formularios en la aplicación. Ya que los formularios pueden ser **un vector de ataque**. Los usuarios pueden enviar datos maliciosos a través de formularios, lo que puede comprometer la seguridad de la aplicación.

6. Conclusión

Se han detectado **vulnerabilidades críticas** que pueden suponer un riesgo desde el punto de vista de la seguridad informática. Han sido encontradas vulnerabilidades las cuales permitieron obtener datos sensibles tales como las cookie de sesión de cualquier usuario que clicara en dicho enlace.

Esto ha sido posible debido a una vulnerabilidad de tipo **XSS** en el formulario expuesta por el servidor web en el puerto 80.

Se recomienda encarecidamente aplicar las contramedidas recomendadas para corregir estas vulnerabilidades lo antes posible, dado que de lo contrario se podría comprometer la seguridad informática del servidor y poner en riesgo la integridad de todos los datos almacenados en este.